Ruolo e obiettivo

Sei un senior full-stack engineer. Costruisci **Organizzatore Dieta**, una web app installabile (PWA) in italiano che trasforma il piano alimentare di un nutrizionista, ricevuto in PDF, in uno strumento d'uso quotidiano: scelta dei pasti, lista della spesa e controllo delle regole del piano. L'uso principale è personale. Alcuni amici la useranno con i loro piani, quindi l'app deve essere multi-utente con dati completamente isolati, senza funzioni social.

Come lavorare

- Parti da un piano di implementazione e aspetta la mia approvazione prima di scrivere codice.
- Procedi per fasi (sotto) e al termine di ciascuna fermati con un riepilogo e come verificarla.
- Scrivi test automatici per l'aggregatore della lista della spesa e per il validatore di regole, che sono la logica critica.
- Non inventare dati nutrizionali. L'app non dà consigli medici né calcola diete, lavora solo sul piano caricato dall'utente.

Stack

Next.js (App Router) + TypeScript, Prisma, PostgreSQL **[Neon / Supabase]**, autenticazione **[Auth.js / Supabase Auth]**, Tailwind, PWA installabile. Deploy su **[Vercel]**. Le chiavi API stanno solo lato server.

Struttura del piano da modellare

Un piano ha una data di inizio e fine. Ogni giorno ha dei pasti (colazione, spuntini, pranzo, cena). Ogni pasto ha una o più opzioni alternative ("oppure"). Ogni opzione è un elenco di ingredienti con quantità e unità (g, ml, pezzi, fette). Il piano contiene anche una sezione di note con **regole** (es. "affettati max 3 volte a settimana, crudo escluso", "pesce almeno 3 volte a settimana", "1–2 pasti liberi a settimana").

Modello dati minimo: `User, Plan, Day, Meal, Option, OptionIngredient, Ingredient (con categoria di reparto), Rule, WeekMenu, WeekMenuEntry`. Ogni tabella legata a un utente ha `userId`, e ogni query lo filtra.

Fase 1: base e lista della spesa

- Autenticazione, un piano inserito a mano (o da JSON di esempio), schermata "menu della settimana".
- Per ogni pasto l'utente sceglie una delle opzioni. Può escludere un pasto (pasto libero).
- Lista della spesa calcolata: somma gli ingredienti per N settimane, raggruppa per reparto, converte in kg/l oltre 1000, ha caselle "comprato" e un pulsante "copia

lista".
- Opzione per aggiungere una quantità fissa di verdure ai pasti principali.
- Funziona offline una volta caricata.

Fase 2: validatore di regole

- Le regole sono dati, non codice: es. `{type: "max_per_week", category: "affettati", max: 3}` e `{type: "min_per_week", category: "pesce", min: 3}`.
- Il validatore controlla il menu settimanale e mostra avvisi chiari ("hai scelto 4 affettati, il piano ne ammette 3") senza bloccare l'utente.
- Test con casi limite (categorie sovrapposte, settimane parziali).

Fase 3: importer PDF

- L'utente carica il PDF. Estrazione server-side con un modello **[specifica modello/API]** capace di leggere anche le immagini delle pagine, restituendo JSON validato con Zod.
- **Schermata di revisione obbligatoria**: l'utente vede cosa è stato letto e può correggere ingredienti, quantità e regole prima del salvataggio. Nessun dato entra nel piano senza revisione.
- Gestisci errori e piani malformati. Limita dimensione del file, numero di import e costo per utente.
- Il PDF originale non viene conservato dopo l'estrazione, salvo scelta esplicita.
- In caso il PDF non sia possibile analizzarlo può esserci la possibiltà di caricare un file JSON adatto per caricare gli alimenti.

Privacy e sicurezza

- Sono dati sulla salute: raccogli il minimo indispensabile, nessuna analytics di terze parti.
- Isolamento per utente verificato con test (un utente non deve mai leggere i dati di un altro).
- Funzione "elimina il mio account e tutti i miei dati". Pagina informativa breve sul trattamento dei dati.
- Rate limiting sulle API e validazione di ogni input.

Design

Mobile-first, pulito, tema chiaro/scuro. Colore principale **[arancione #F7931E]**. Priorità: aprire l'app e vedere subito il menu di oggi.

Fuori scopo (non farlo)

Tracking di peso o calorie, ricette, funzioni social, notifiche push, pagamenti.

Consegna

README con setup locale, variabili d'ambiente e istruzioni di deploy, più un `CONTEXT.md` con le decisioni architetturali prese.

Un consiglio operativo: dopo la Fase 1 usa l'app per una settimana prima di far partire le altre. Ti dirà se il flusso di scelta dei pasti funziona davvero, prima di investire nell'importer, che è la parte più costosa e fragile.